

Uber :

Functional requirements:

user should be able to book a cab based on his preference,
user should be able to search for locations based on his preference,
Rider should be able to accept/reject any ride based on his preference

Non-functional requirements:

data-partitioning,
load balancing,
fault tolerance,
vertical scalability,
horizontal scalability,
low latency,
high throughput,

CAP theorem (Here we need to Strong Consistency in order to avoid double Booking/double payment Scenario & for Search scenario we need eventual consistency is fine but here we need 99.99% uptime i.e Availability>>Consistency)

Core Entities:

user,
booking,
payment,
history

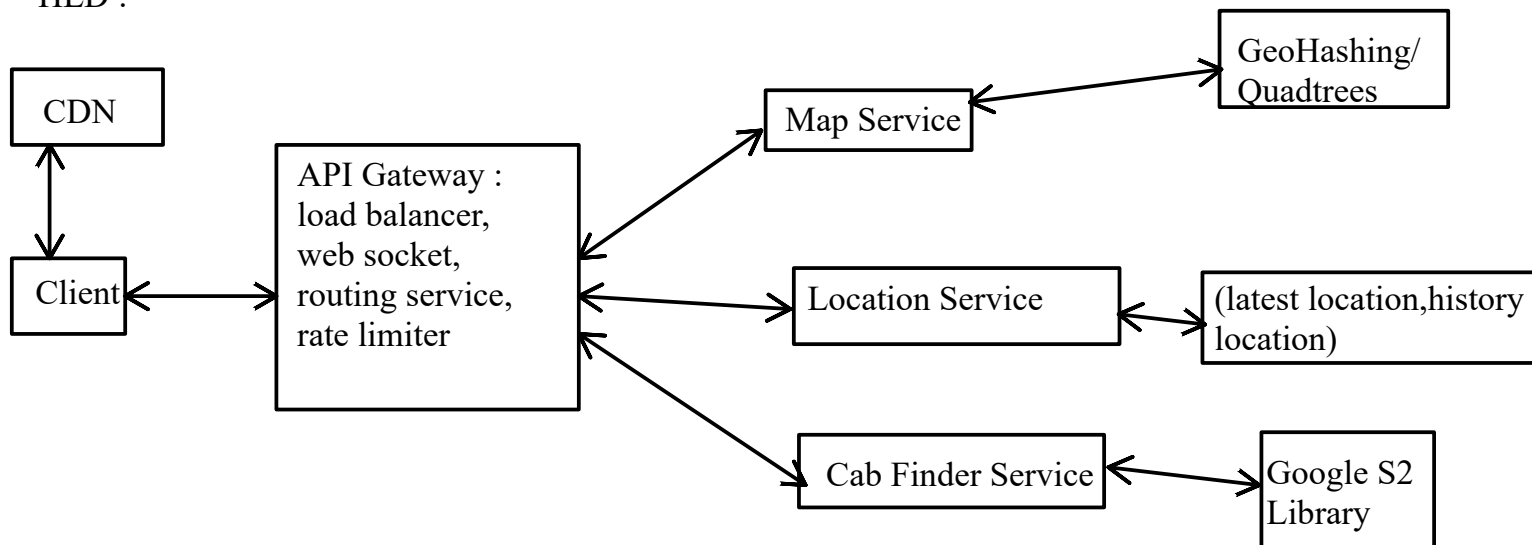
API :

POST:/cab/booking

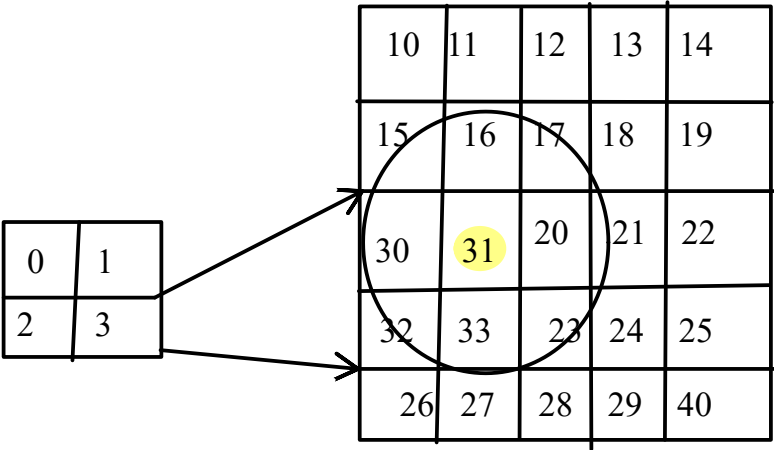
```
{  
  userID,  
  bookingMetadata  
}
```

GET:/search/{locationID}

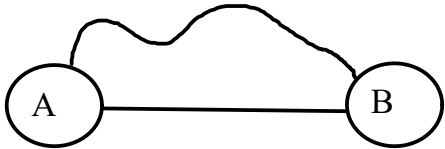
HLD :



GeoHashing
(lat,long) -> chunkID



Google S2



We cannot use Euclidian distance,
we need to use Google S2 lib

Consistent Hashing,
gossip protocol